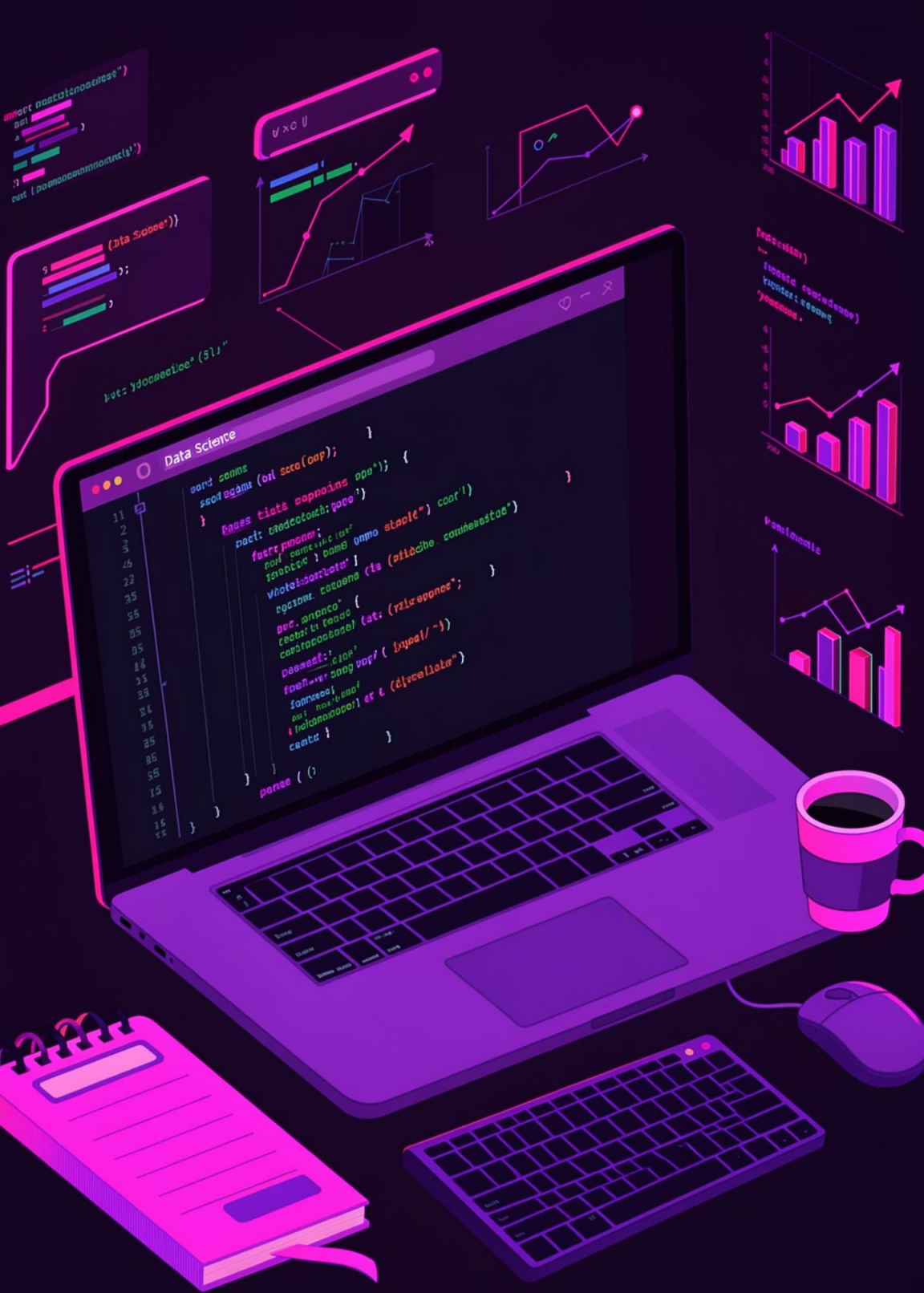




Jupyter: Pandas Inicial

Carga de datos, gestión de valores Nulos y Duplicados

El Poder de Jupyter y Pandas

Jupyter Notebooks

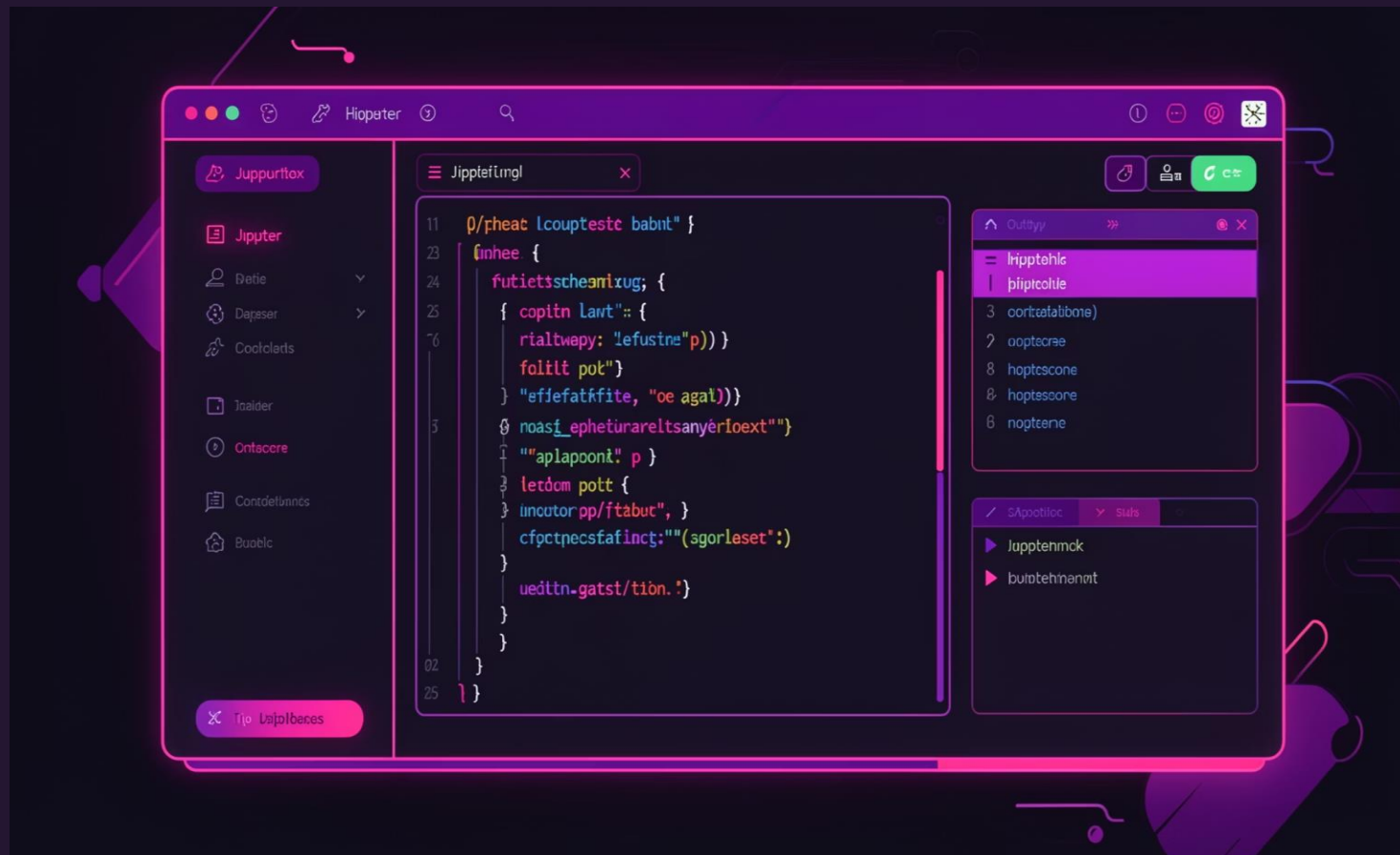
Entorno interactivo que combina código ejecutable, visualizaciones y texto explicativo en una sola interfaz.

Pandas

Librería de Python especializada en manipulación y análisis de datos, con estructuras como DataFrames.

Objetivo Principal

Preparar datos para análisis, identificando y corrigiendo errores comunes que afectan la calidad.



Paso 1: Carga de Datos en Pandas

Pandas ofrece funciones versátiles para importar datos de múltiples formatos.

CSV

```
pd.read_csv()
```

Archivo de texto con valores separados por comas

Excel

```
pd.read_excel()
```

Libros de cálculo de Microsoft Excel

JSON

```
pd.read_json()
```

Datos estructurados en formato JavaScript

```
import pandas as pd
df = pd.read_csv('datos.csv')
df_copia = df.copia() #Copia del fichero
print(df.head())
```

Ejemplo básico: Carga un archivo CSV y muestra las primeras 5 filas para verificar el contenido. Realizamos una copia de seguridad.

Paso 2: Identificando Valores Nulos (NaN)

¿Qué son los valores nulos?

Datos faltantes que pueden sesgar el análisis y producir resultados incorrectos. Pandas usa **NaN** (Not a Number) para datos numéricos y **None** para no numéricos.

Métodos de Detección

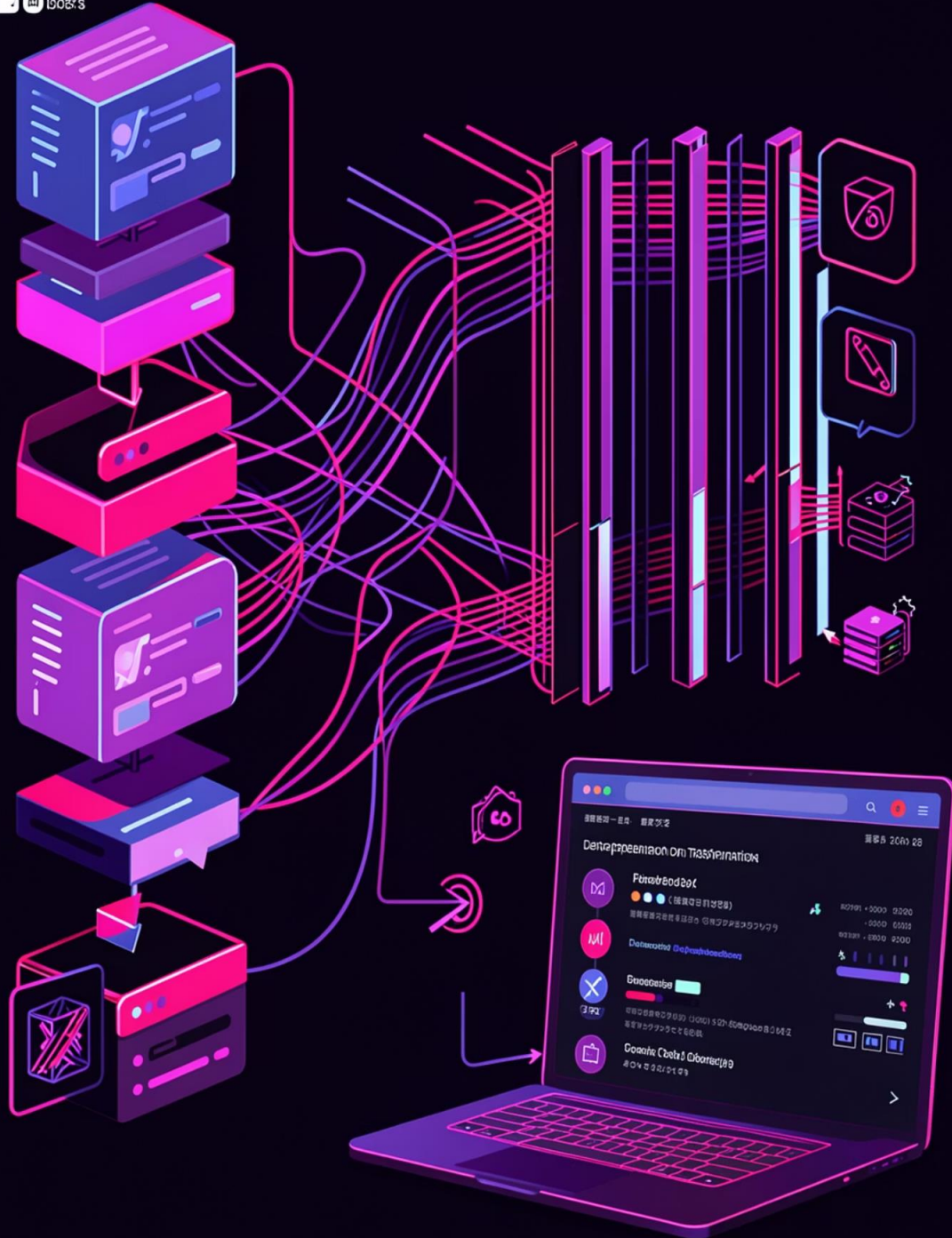
- `df.isnull()`: DataFrame booleano con `True` en posiciones nulas
- `df.isna()`: Alias equivalente a `isnull()`
- `df.isnull().sum()`: Suma de nulos por columna



```
print(df.isnull().sum()) # Muestra conteo de nulos por columna
```


Data Cleaning

100%



Paso 3: Estrategias para Manejar Valores Nulos

1

Eliminación de Filas

```
df.dropna()
```

Elimina filas con al menos un valor nulo. Rápido pero puede perder información valiosa.

2

Eliminación de Columnas

```
df.dropna(axis=1)
```

Elimina columnas con al menos un valor nulo. Útil cuando la columna tiene muchos nulos.

3

Imputación con Valor Constante

```
df.fillna(0)
```

Rellena nulos con un valor específico como 0, -1 o "Desconocido".

4

Imputación con Estadísticos

```
df['col'].fillna(df['col'].mean())
```

Rellena con media, mediana o moda de la columna para preservar distribución.



Precaución: La eliminación puede perder información valiosa. Considera el contexto antes de aplicar cada estrategia.

Paso 4: Identificando Filas Duplicadas

¿Por qué son un problema?

- Inflan conteos y métricas agregadas
- Distorsionan promedios y estadísticos
- Sesgan modelos de machine learning
- Producen análisis incorrectos



Detección Básica

`df.duplicated()` devuelve Serie booleana indicando duplicados



Visualización

`df[df.duplicated()]` muestra las filas duplicadas completas



Conteo

`df.duplicated().sum()` número total de filas duplicadas

Paso 5: Eliminando Filas Duplicadas

La solución directa para eliminar duplicados y mantener datos únicos en tu conjunto.

1

Básico

```
df.drop_duplicates()
```

Elimina filas idénticas en todas las columnas

2

Por Columnas

```
subset=['col1', 'col2']
```

Elimina basados en columnas específicas

3

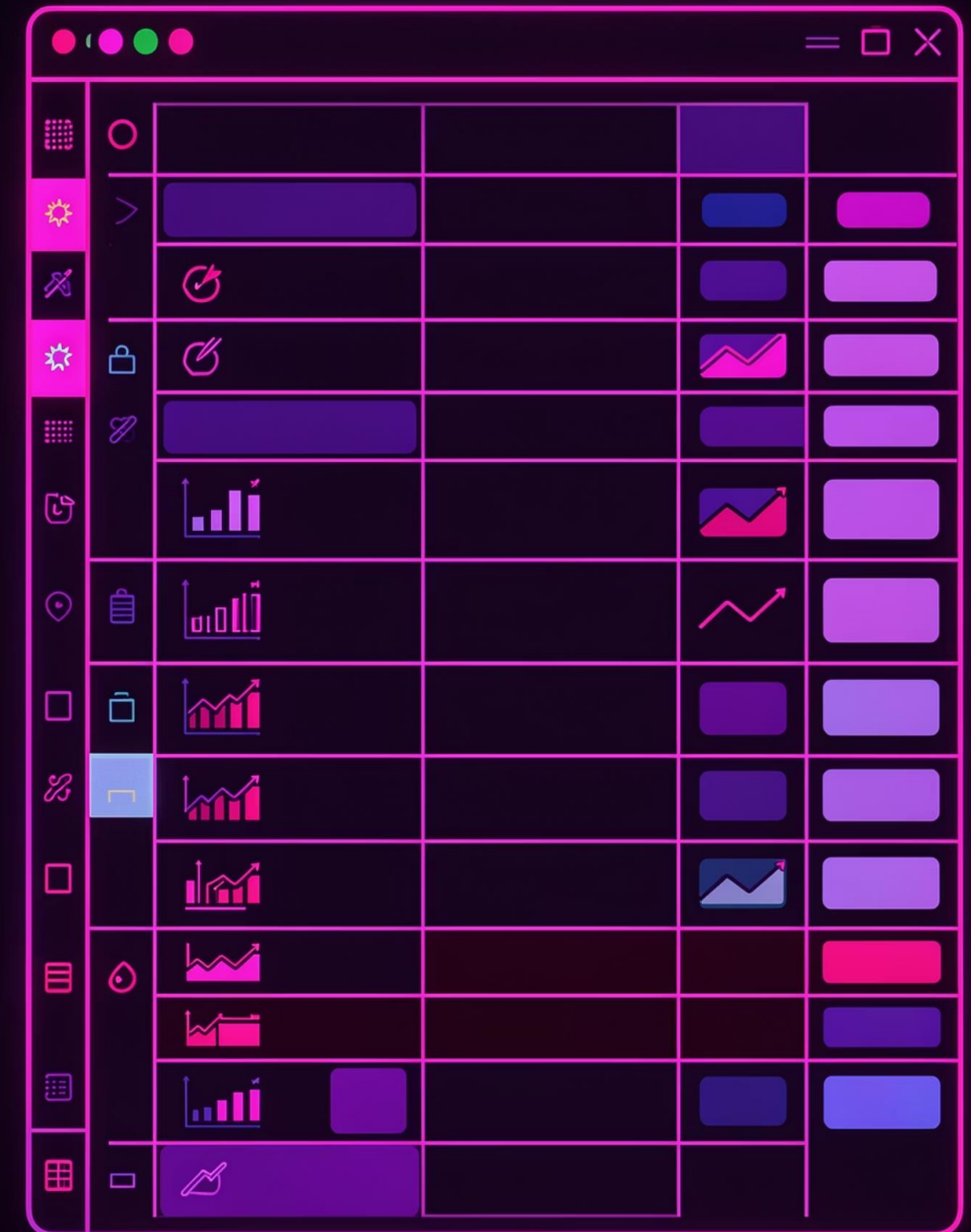
Conservar Último

```
keep='last'
```

Mantiene la última ocurrencia

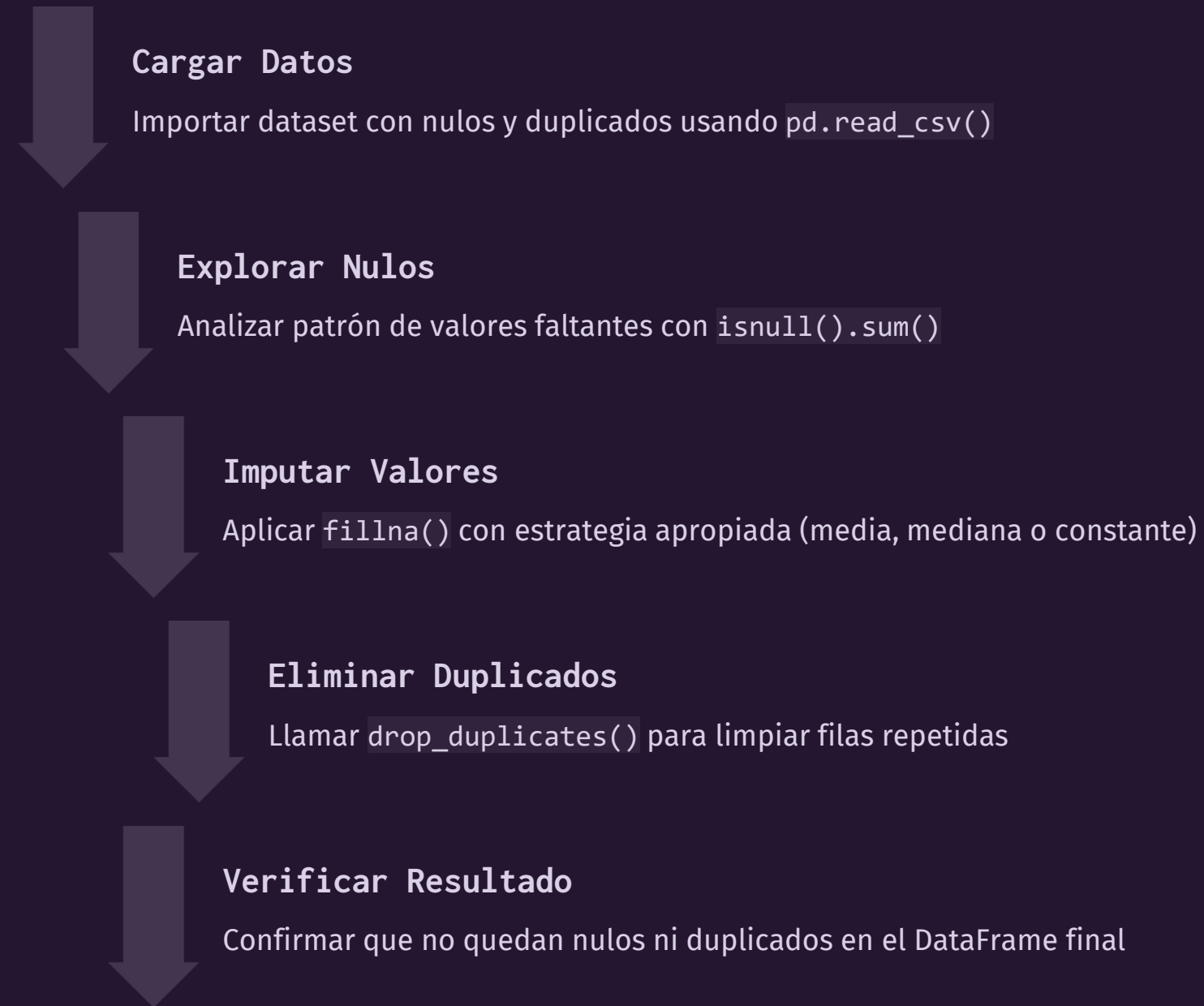
```
df_limpio = df.drop_duplicates()
print(f"Filas originales:
{len(df)}")
print(f"Filas limpias:
{len(df_limpio)}")
```

El método retorna un nuevo DataFrame sin modificar el original. Usa `inplace=True` para modificar el DataFrame actual.



Ejemplo Práctico: Uniendo Conceptos

Vamos a aplicar todas las técnicas aprendidas en un flujo completo de limpieza de datos.





Cheatsheet — Referencia Rápida

Carga

```
pd.read_csv('f.csv')
```

```
pd.read_excel('f.xlsx')
```

```
pd.read_json('f.json')
```

```
pd.read_sql(query, conn)
```



Explorar

```
df.shape
```

```
df.info()
```

```
df.describe()
```

```
df.head(n) /  
df.tail(n)
```

```
df.dtypes / df.columns
```



Nulos

```
df.isnull().sum()
```

```
df.dropna()
```

```
df.dropna(how='all')
```

```
df.fillna(valor)
```

```
df['c'].fillna(df['c'].mean())
```



Duplicados

```
df.duplicated().sum()
```

```
df[df.duplicated()]
```

```
df.drop_duplicates()
```

```
df.drop_duplicates(keep='last')
```

```
df.drop_duplicates(keep=False)
```

Buenas Prácticas y Consideraciones



Automatización

Encapsula la limpieza en funciones reutilizables para aplicar el mismo proceso a múltiples datasets.



Contexto

La mejor estrategia (eliminar vs. imputar) depende del dataset, tipo de datos y objetivo del análisis.



Validación

Siempre verifica el resultado de la limpieza comparando dimensiones y estadísticos antes y después.

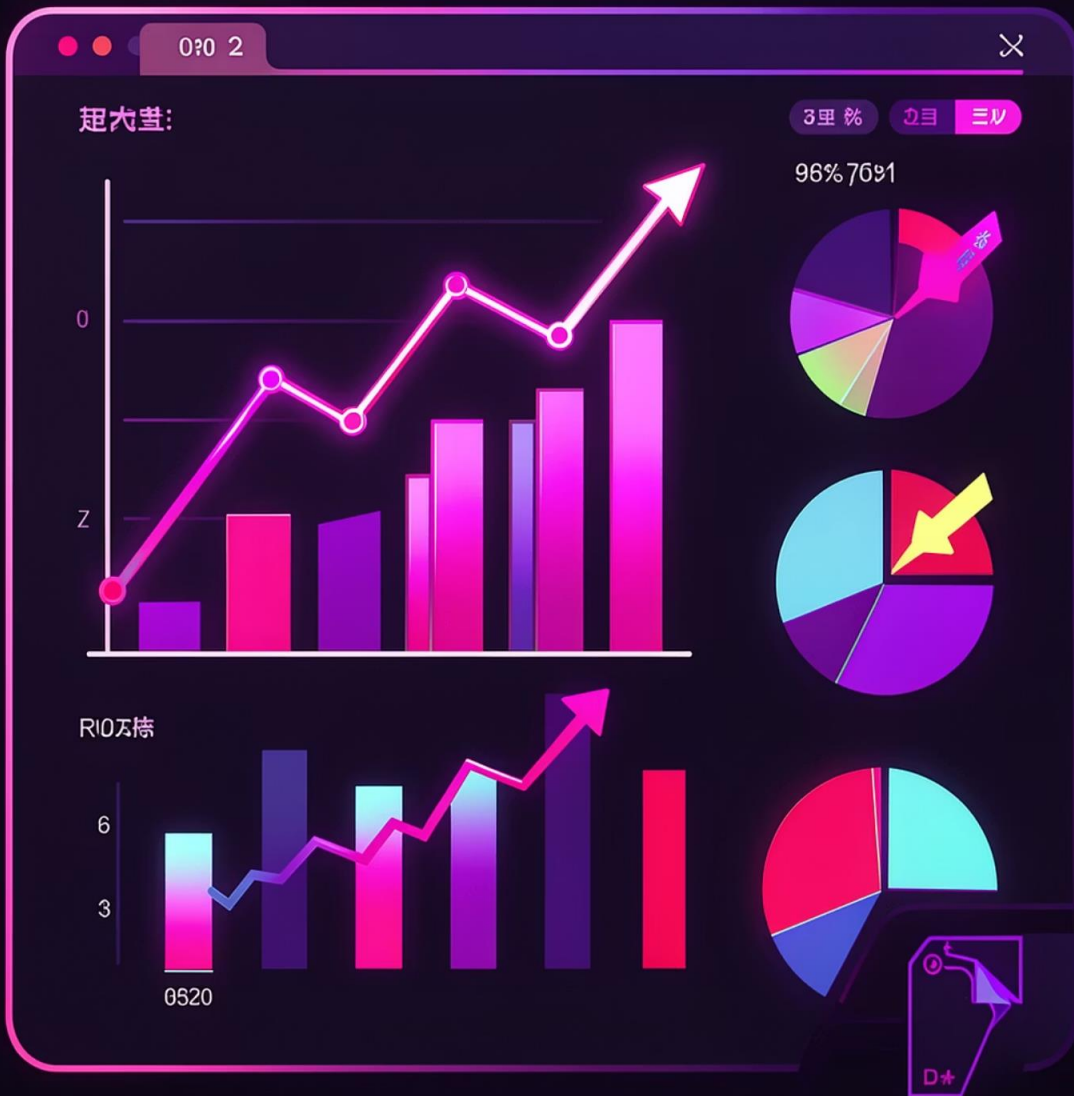


Documentación

Comenta tu código para entender el proceso de limpieza y justificar las decisiones tomadas.

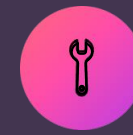
 Consejo: Guarda una copia del dataset original antes de aplicar transformaciones irreversibles como eliminación de filas.

Conclusión: Datos Limpios, Análisis Confiables



Fundamental

La carga, gestión de nulos y duplicados son pasos esenciales para cualquier análisis de datos.



Potentes

Pandas y Jupyter ofrecen herramientas completas y flexibles para realizar estas tareas eficientemente.



Resultados

Domina estas técnicas para obtener insights precisos y confiables que impulsen decisiones correctas.

La calidad de tu análisis depende directamente de la calidad de tus datos. ¡Invierte tiempo en la limpieza para obtener resultados confiables!